

Lecture 1 Overview and Recursion

What is Data

The set of symbols which can be recognized, stored and processed by the computer.

Basic Concepts about Data

- Data Object:
abstract representation of real-world things about which data is collected. (Information of all students)
- Data element:
a specific one of a data object. (The information of one student)
- Data item:
smallest, indivisible unit that constitute a data element. (age, name and so on)

What is a Data Structures

The organization of data and its associated operations

basic operations includes insertion, deletion, traversal(iteration), search, update, sorting and merging and so on.

Why So Many Data Structures

- Ideal Data Structure:
elegant, fast, memory efficient
- Generates tensions:
 - time vs space
 - one operation's performance vs another's
 - generality vs simplicity

Why do we need to learn data structures

- The way to organize data affects the performance of a program for different tasks
- Developing efficient algorithms in all phases of advanced data processing and computer science.

Class Overview

- Introduction of the basic data structures used in computer software.

- linear data structures: array, stack, queue
- nonlinear data structures: tree, graph, set
- fundamental data structures
- logical structure and storage structure
- time-space complexity analysis
- their implements
- Introduction of implements and analysis of sorting and searching algorithms.

Recursion

Overview

- Definition of Recursion
- Conditions for Recursion to Work
- How Recursion Works Internally
- Tower of Hanoi
- Binary Search

Definition of Recursion

- A function is said to be **recursive** if it is defined in terms of **itself**
- **Recursion** is a programming technique that enables the solution to certain problems to be expressed in terms of solutions to **smaller instances** of the problems **itself**.

Conditions for Valid Recursion

- A **base case** where the input value or size is the **smallest possible** and it is **non-recursive**.
- The input of every recursive call must be **smaller in value or size than the input of original function**.

Design Rule: Assume all the recursive calls work

- do **not** concern yourself how recursion is unfolding during execution

Tower of Hanoi

- $n > 1$ disks to be moved from A to C using B.
- move top $n - 1$ disks from A to B using C
- move disk 1 from A to C using B
- move top $n - 1$ disks from B to C using A

Recursive Solution

- `Hanoi(n)` when $n = 1$;
- `Hanoi(n,A,B,C) = Hanoi(n-1,A,C,B) + Hanoi(1,A,B,C) + Hanoi(n-1,B,A,C)`

$$H(n) = 2 * H(n-1) + 1;$$

Binary Search

Input:

- Sorted array of size n
- Value b to be searched for

Output:

- If b is found, return the corresponding index
- If not, return -1

Method is recursive:

- Compare b with `x[mid]`
- If $b = x[mid]$, return mid.
- If less, find in the left half.
- If larger, find in the right half.